

Portfolio Pulse

Project Presentation & Implementation Guide

CLEAR DIRECTION FOR EVERY HOLDING

A secure, plain-language portfolio analysis experience that turns CSV holdings data into understandable performance, risk, diversification, and ticker-level insights.

PROJECT

PORTFOLIO PULSE

PRESENTATION DATE

AUGUST 16, 2026

Presentation Guide

01	Project Outline	Purpose, audience, workflow, architecture, feature set, and security posture.
02	Sample Data & Demo	Input schema, representative sample data, video placeholder, and application screenshots.
03	Prompts	The initial planning prompt and the major iterative enhancement prompts that shaped the product.
04	How To Run the App	Prerequisites, installation, secure LLM configuration, startup, tests, and troubleshooting.

01 Project Outline

From portfolio exports to clear, educational insights.

Project Goal

Portfolio Pulse helps everyday investors understand what changed, why it changed, and what deserves attention—without presenting personalized investment advice.

Build a responsive application that accepts one or more portfolio CSV exports, validates and sanitizes the data, calculates portfolio and ticker performance across multiple time ranges, and explains results in plain language.

Target Users

- **Individual investors** — people who want understandable portfolio feedback without finance-heavy terminology.
- **Students and reviewers** — users evaluating portfolio analytics, secure data handling, and iterative product development.
- **Portfolio learners** — users who want to explore diversification, concentration, performance drivers, and historical risk.

End-to-End Workflow

1. Accept the disclaimer before entering the application.
2. Upload one or more validated portfolio CSV files.
3. Retrieve bounded market history and metadata for supported tickers.
4. Calculate portfolio, source, industry, and ticker-level performance.
5. Present plain-language summaries, interactive charts, and downloadable PDF reporting.
6. Optionally ask the secure Pulse assistant questions about normalized analysis metrics.

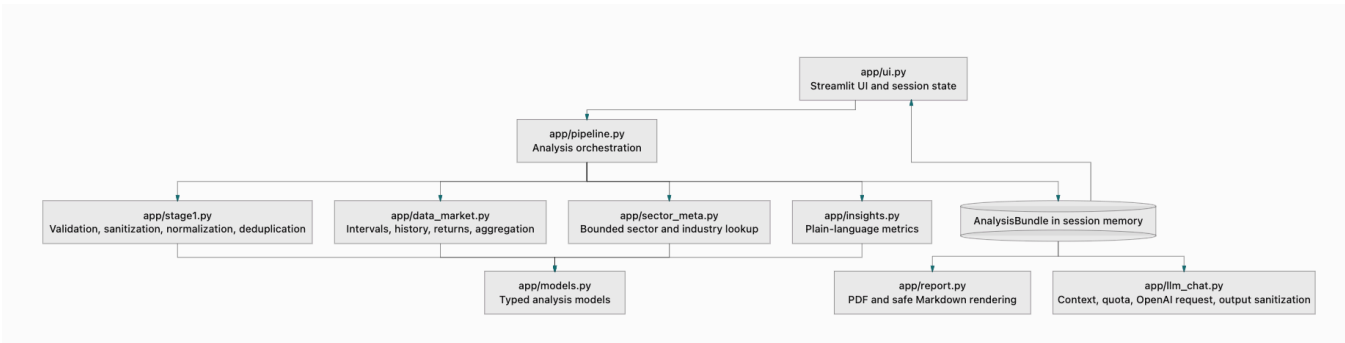
Core Experience

Area	What it provides
Data Overview	Upload expansion, holdings summary, positions by amount paid, and overall investment timeline.
Performance	Portfolio value, dollar and percentage change, contributors/ detractors, diversification, concentration, swinginess, drawdown, source and industry comparisons.
Ticker Details	Current price, average price paid, position information, return trends, and ticker-level investment timeline.
Ask Pulse	Server-side LLM assistant grounded only in normalized portfolio metrics with explicit data-transfer consent.
PDF Report	Downloadable, presentation-friendly summary with portfolio, source, and industry comparisons.

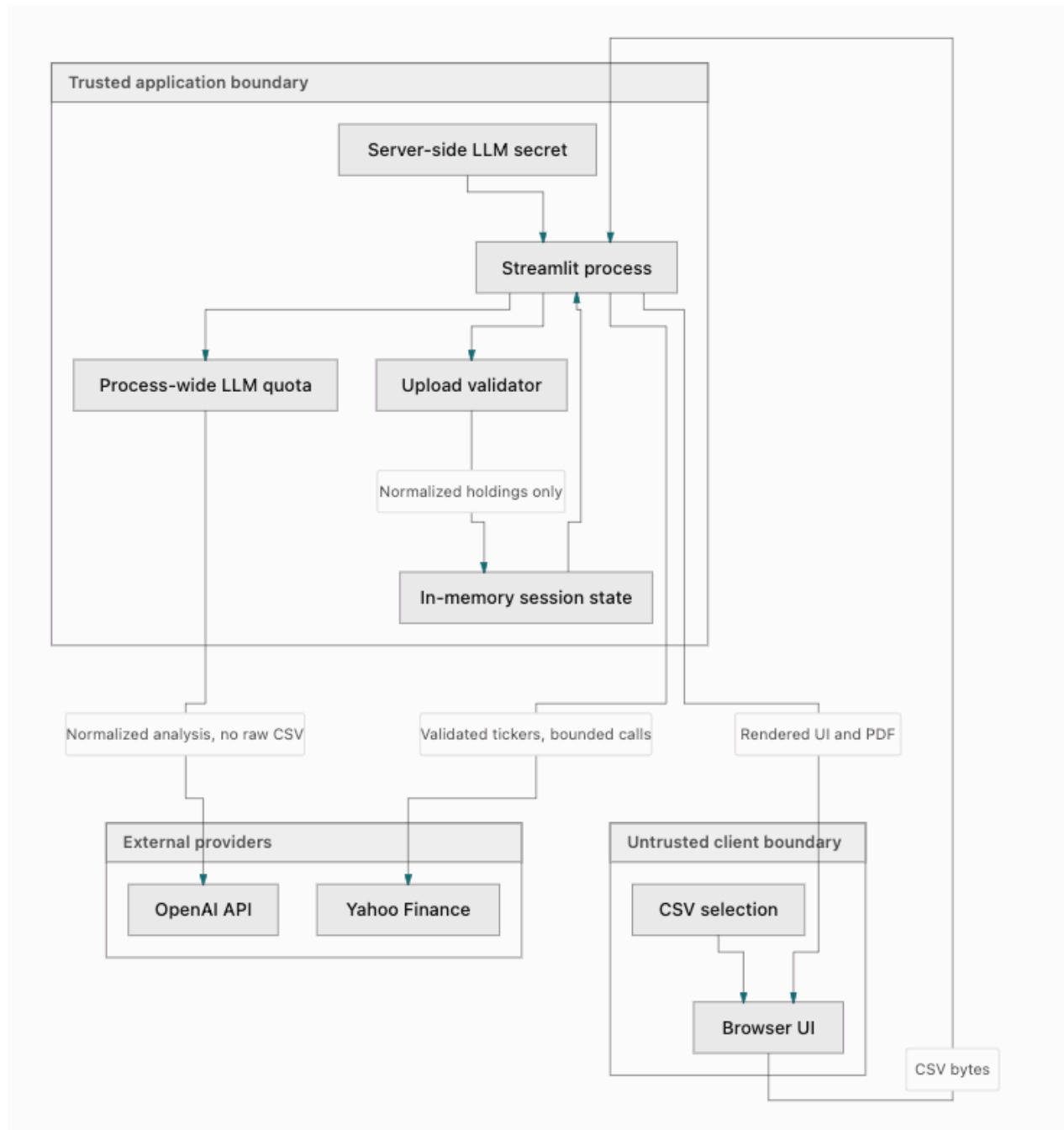
Technology & Architecture

- **Application** — Python 3.12 and Streamlit.
- **Data processing** — pandas, NumPy, Decimal-based holdings normalization, and yfinance market data.
- **Visualization** — Streamlit-native charts and Altair.
- **Reporting** — ReportLab PDF generation.
- **Assistant** — OpenAI Responses API through server-side credentials only.
- **Quality** — pytest unit/security tests and Playwright cross-viewport end-to-end tests.

Component Architecture



Deployment and Trust Boundaries



Security by Design

- UTF-8 CSV only; maximum 2 MB per file, 10 files, 10 MB combined, 2,000 rows, and 100 unique tickers.
- Rejects spreadsheet formulas, macros, macro-like executable content, binary payloads, and unsupported extensions.

- Masks sensitive data, discards unsupported free-text columns, and replaces original filenames with non-identifying labels.
- Pins the OpenAI endpoint, keeps API keys server-side, sanitizes model output, applies quotas, and requires explicit consent.
- Bounds and caches outbound market-data requests; enables XSRF and CORS protections.
- Uses exact dependency pins, a hashed lockfile, and a known-vulnerability audit.

Current validation status

39 Python tests passed, 6 Playwright tests passed across phone/tablet/desktop, and the locked dependency audit reported no known vulnerabilities. Authentication was intentionally outside the requested scope.

02 Sample Data & Demo

Synthetic holdings demonstrate both a broad portfolio and multi-lot ticker history.

Accepted CSV Structure

Column	Status	Purpose
ticker	Required	Trading symbol, normalized to a strict safe format.
avg_purchase_price	Required	Approximate average price paid per share.
shares	Required	Positive number of shares held.
purchase_date	Optional	Lot purchase date used in investment timelines.
currency	Optional	Currency label; USD is used in the samples.
security_type	Optional	Allowlisted type such as Stock, ETF, REIT, Fund, Bond, Cash, or Option.

Sample Dataset A — Broad Portfolio

sampledata.csv contains a diversified set of stocks and funds with purchase dates, currencies, and security types.
Representative rows:

Ticker	Avg. price paid	Shares	Purchase date	Currency	Type
AAPL	172.35	12	2021-03-14	USD	stock
MSFT	338.20	8	2022-07-09	USD	stock
NVDA	865.10	4	2023-02-21	USD	stock
AMZN	122.55	15	2020-10-05	USD	stock
GOOGL	140.90	20	2021-06-17	USD	stock

Sample Dataset B — Multiple Tax Lots

sampledata2.csv repeats tickers across purchase dates to demonstrate lot-level history and how recent additions influence the analysis. Representative rows:

Ticker	Avg. price paid	Shares	Purchase date	Currency	Type
AAPL	172.35	12	2021-03-14	USD	stock
AAPL	176.80	20	2022-01-18	USD	stock
AAPL	181.25	15	2023-06-07	USD	stock
MSFT	338.20	8	2022-07-09	USD	stock
MSFT	365.15	6	2024-05-13	USD	stock

Video Demo

GOOGLE DRIVE VIDEO DEMO

https://drive.google.com/file/d/1ofq7SyfFd9k6J0nF3aOpE8-eY4Mt9QUV/view?usp=share_link

Application Screenshots

Deploy 

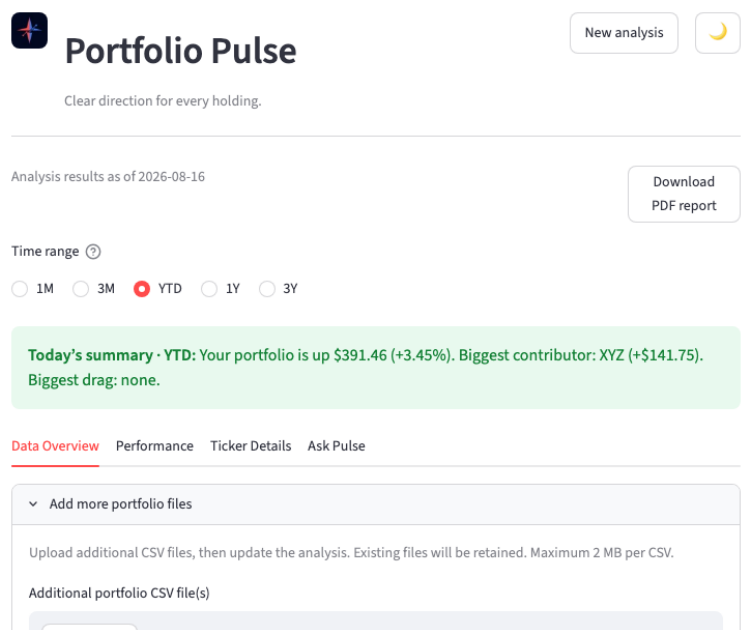
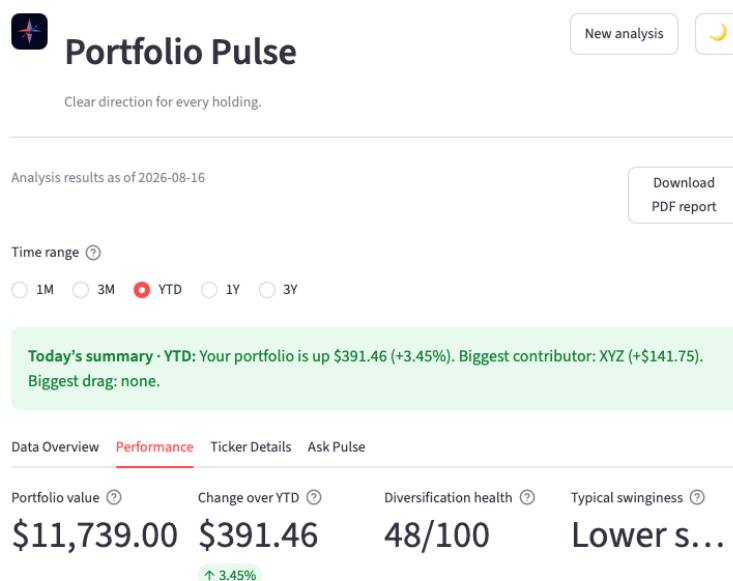


Figure 1 — Data Overview

The results header, time-range controls, Today's summary, PDF download, and add-more-files workflow.



What drove the change over YTD?

Figure 2 — Performance

Portfolio value, dollar and percentage change, diversification health, and plain-language risk framing.

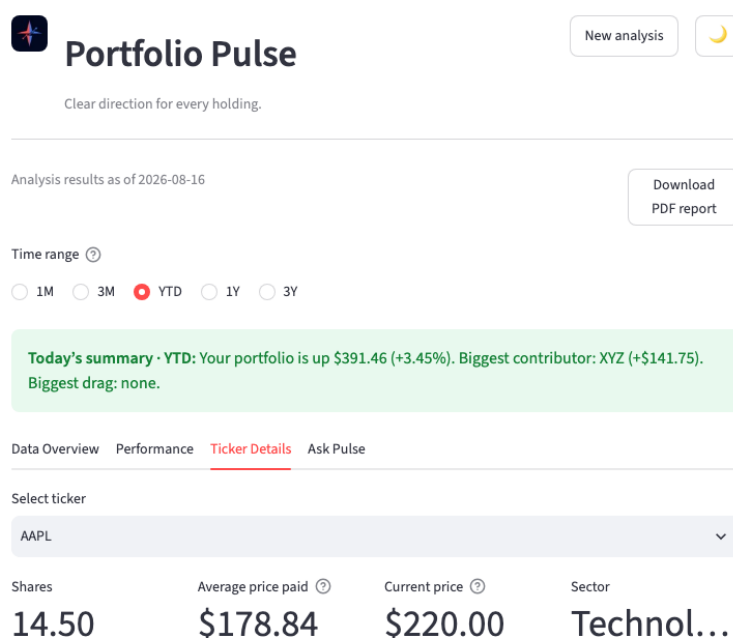


Figure 3 — Ticker Details

Ticker selection, shares, average price paid, current price, and company classification at a glance.

03 Prompts

The product evolved through a plan-first prompt followed by targeted, testable enhancement requests.

Prompting approach

Start with requirements and an implementation plan, then iterate in small product slices: reliability, experience, analytics, reporting, assistant integration, and security.

00

PRD generation prompt

“Assume the role of an expert technology and product partner brainstorming the following idea. An application where primary user personal is that of a beginner to intermediate investor. The user typically holds multiple trading/investment accounts and is having a hard time trying to reconcile the transactions and investments and track performance across accounts. The users will provide one or more csv files (at least one per source) and the app will reconcile all details from every source. Once analysis is complete, the user gets to see a performance overview of the entire portfolio, is able to drill down the performance by each stock ticker. As a start the user has the ability to chat with the entire portfolio analysis using an llm chat interface. Remember the application is a simple performance tracker and risk awareness tool. The intent is not to provide any guidance or advice on investment or recommendations on managing their portfolio. This is a disclaimer every user will be required to sign. The outcome of this exercise is a markdown file that can be consumed by a coding AI agent to generate code and test scenarios for the application.”

01

Initial planning prompt

“Review the PRD requirements and come up with a plan for implementation. Start with assessing software availability and then next steps. Capture the plan or todo in a separate Plan.md document.”

Result: Established the phased implementation plan, architecture, dependencies, and acceptance criteria.

02

Plan implementation

“Implement the plan as specified. Do not edit the plan file itself. Mark the existing todos as in progress as you work, starting with the first one, and continue until all todos are complete.”

Result: Produced the initial working Streamlit application and automated tests.

03

Theme and launch disclaimer

“Add a dark mode option to the app. Show the disclaimer on launch and do not allow the user to proceed unless they agree that they have read it.”

Result: Added native light/dark theming and a launch gate.

04

Information architecture

“Place the data on separate tabs named Data Overview, Performance, and Ticker Details. Drop sentiment analysis and include charts on each tab.”

Result: Replaced stage-oriented output with user-oriented navigation.

05

Brand and readability

“Replace source_id with readable filenames, humanize underscored column names, remove low-value sidebar content, move dark mode to the header, and give the app a name and logo.”

Result: Created the Portfolio Pulse brand, sticky header, readable labels, and simplified navigation.

06

Charts and PDF reporting

“Use pie charts for positions and portfolio composition. Replace Markdown download with PDF. Include source comparison, industry comparison, and net growth in the report.”

Result: Improved visual communication and delivered a polished PDF report.

07

Incremental analysis

“Add an option on Data Overview to upload more files and update the existing analysis.”

Result: Enabled additive portfolio analysis without restarting.

08

Plain-language insights

“Make the app answer “what does this mean for me?” Add Today’s summary, plain-language metrics, diversification and risk indicators, performance drivers, filters, dollar changes, and safe non-advisory suggestions.”

Result: Shifted the product from charts-only output to understandable portfolio guidance.

09

Ticker-level detail

“Move crowded investment timelines to ticker level, show overall portfolio investment at portfolio level, and include current stock price on Ticker Details.”

Result: Improved chart readability and ticker context.

10

LLM assistant

“Include a chat window connected to an LLM so users can ask questions about the analysis. Keep API calls and API keys secure.”

Result: Added Ask Pulse with server-side configuration, grounded context, consent, quotas, and user-friendly errors.

11

Security hardening

“Address all security issues except authentication. Limit each CSV to 2 MB, reject macros and formulas, and remove application test mode.”

Result: Added strict ingestion controls, network bounds, LLM hardening, dependency locking, and isolated test fixtures.

Prompt Design Lessons

- State the user outcome first, then provide concrete acceptance criteria.
- Use exact labels and interaction expectations when UI wording matters.
- Separate large enhancements into analytics, presentation, integration, and security concerns.
- Pair every behavioral request with validation expectations across unit and browser tests.
- Request user-friendly errors explicitly; provider errors should never become customer-facing technical messages.

04 How To Run the App

The following instructions are adapted from README.md.

Prerequisites

- Python 3.11 or newer; Python 3.12 is recommended.
- A terminal opened at the project root.
- Node.js 18 or newer only if Playwright end-to-end tests will be run.
- An OpenAI API key only if Ask Pulse will be enabled.

1. Create the Environment

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install --require-hashes -r requirements.lock
cp .env.example .env
```

The hashed lockfile installs the exact reviewed dependency tree used by the project.

2. Configure Ask Pulse Securely (Optional)

Use a deployment environment variable whenever possible:

```
export LLM_API_KEY="YOUR_OPENAI_API_KEY"
export LLM_MODEL="gpt-4.1-mini"
```

OPENAI_API_KEY is also supported. For local Streamlit development:

```
cp .streamlit/secrets.toml.example .streamlit/secrets.toml
```

- Replace the placeholder only in .streamlit/secrets.toml.
- Never commit .env, secrets.toml, real keys, credentials, or keys embedded in CSV files.
- The browser never receives the provider key; LLM calls run on the Streamlit server.

3. Start the Application

```
source .venv/bin/activate
streamlit run run_app.py
```

Alternative entry point:

```
streamlit run app/ui.py
```

Expected result

Streamlit opens Portfolio Pulse in your default browser. Review and accept the disclaimer, then upload one or more compliant CSV files.

4. Upload Data

- Required columns: ticker, avg_purchase_price, shares.
- Optional columns: purchase_date, security_type, currency.
- Use UTF-8 CSV files no larger than 2 MB each.
- Do not include formulas, macros, macro-like executable content, or unsupported free-text fields.
- Use the “Add more portfolio files” control on Data Overview to extend an existing analysis.

5. Run Tests

Unit and security tests:

```
pytest tests/ -q
```

End-to-end browser tests:

```
cd e2e
npm install
npx playwright install chromium
npx playwright test
```

Troubleshooting

Symptom	What to check
No module named app	Activate the virtual environment and run: <code>pip install -e .</code>
Pulse is offline	Confirm the server-side API key is configured, valid, and has available quota.
CSV rejected	Confirm UTF-8 encoding, .csv extension, required columns, size limits, and absence of formulas/macros.
No market metrics	Confirm tickers use a supported format and that the server can reach market-data services.
Stale interface	Stop Streamlit and restart it; deployment auto-reload is intentionally disabled.